

INSTITUTE OF SCIENCE, TECHNOLOGY & ADVANCED STUDIES (VISTAS)
(Deemed to be University Estd. u/s 3 of the UGC Act, 1956)
PALLAVARAM, THALAMBUR, PERIYAPALAYAM, THIRUVANMIYUR, ADYAR - CHENNAI

CROWD DETECTION IN A BUS – REAL-TIME PASSENGER DETECTION & COUNTING USING YOLO OBJECT DETECTION

DHARAN C – 24619109

ANTO X – 24619158

VARUN S – 24619155

Under the guidance of

S. DIVYA BAIRAVI, M.E., (Ph.D)

Assistant Professor, Department of CSE (AI & ML)

VELS INSTITUTE OF SCIENCE, TECHNOLOGY & ADVANCED STUDIES (VISTAS)

MINI PROJECT REPORT

Submitted to the

SCHOOL OF ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (AI & ML)

in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING (AI & ML)

VELS INSTITUTE OF SCIENCE, TECHNOLOGY AND ADVANCED STUDIES (VISTAS)

CHENNAI – 600 117

OCTOBER 2026

BONAFIDE CERTIFICATE

Certified that this project report titled "CROWD DETECTION IN A BUS – REAL-TIME PASSENGER DETECTION & COUNTING USING YOLO OBJECT DETECTION" is the bonafide work of **Dharan C (24619109)**, **Anto X (24619158)** and **Varun S (24619155)** who carried out the project work under my supervision. Certified further that, to the best of my knowledge, the work reported here does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE OF THE HOD

Dr. K. Ulaga Priya, M.E., Ph.D.
Associate Professor & Head
Department of CSE (AI & ML)
School of Engineering
VISTAS
Chennai – 600 117

SIGNATURE OF THE SUPERVISOR

S. Divya Bairavi, M.E., (Ph.D)
Assistant Professor
Department of CSE (AI & ML)
School of Engineering
VISTAS
Chennai – 600 117

Submitted for the Project Work and Viva-Voce examination held on …… at Vels Institute of Science, Technology and Advanced Studies (VISTAS), Chennai – 600 117.

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

It is certainly a great delight and proud privilege to acknowledge the invaluable support, guidance, and encouragement received from numerous individuals throughout the development and completion of this project.

First and foremost, we express our profound gratitude and heartfelt thanks to our respected FOUNDER & CHAIRMAN, Shri. Dr. ISHARI K. GANESH, for providing outstanding infrastructural facilities, laboratory resources, and a stimulating academic environment that fostered our technical growth.

We convey our sincere thanks and deep gratitude to our HEAD OF THE DEPARTMENT, Dr. K. Ulaga Priya, for continuous support, constructive feedback, administrative assistance, and encouragement during all phases of this project.

We owe our deepest gratitude and warmest thanks to our esteemed PROJECT SUPERVISOR and GUIDE, S. Divya Bairavi, M.E., (Ph.D), Assistant Professor, Department of CSE (AI & ML), for her invaluable guidance, meticulous reviews, insightful suggestions, and unwavering encouragement throughout the conceptualization, system design, prototype implementation, and report preparation. We are thankful to our Project Coordinator **Ms. S. M. Sowmiya**.

We also extend our sincere appreciation to the Faculty Members, Technical Staff, and Laboratory Staff of the Department of Computer Science and Engineering (AI & ML), whose constant assistance and technical advice were instrumental in overcoming experimental challenges.

Last but never least, we express our heartfelt love and indebtedness to our beloved parents and friends for their unconditional moral support, patience, and constant motivation, without which this endeavor would not have been possible.

DHARAN C (24619109)
ANTO X (24619158)
VARUN S (24619155)

TABLE OF CONTENTS

CHAPTER NO.	TABLE OF CONTENTS	PAGE NO.
	ABSTRACT	ii
	LIST OF FIGURES	iii
	LIST OF TABLES	iv
	ABBREVIATIONS	v
1	**INTRODUCTION**	1
	1.1 Overview of the Project	1
	1.2 Scope and Objective	3
2	**REVIEW OF THE LITERATURE**	6
	2.1 Literature Overview	6
	2.2 Research Summary	6
3	**SYSTEM DESIGN**	10
	3.1 Existing System	10
	3.2 Proposed System	10
	3.3 System Architecture	11
	3.4 Data Flow / Workflow	12
	3.5 Hardware Requirements	15
	3.6 Software Requirements	16
4	**IMPLEMENTATION**	18
	4.1 Methodology	18
	4.2 Implementation	20
	4.2.1 Image Capture & Trigger Module	21
	4.2.2 Person Detection Module (YOLOv8)	21
	4.2.3 Crowd Classification Module	21
	4.2.4 Flask Backend & Dashboard Module	22
	4.2.5 Data Storage & Live Update Module	22
	4.3 Integration and Deployment	23
5	**RESULTS, CONCLUSION AND FUTURE WORK**	24
	5.1 Results and Evaluation	24
	5.2 Discussion	26
	5.3 Challenges and Limitations	26
	5.4 Conclusion	27
	5.5 Future Work	28
	5.6 Publication Outcome	28
	REFERENCES	30
	APPENDIX A – SOURCE CODE	33
	APPENDIX B – CONFERENCE ACCEPTANCE LETTER	39

ABSTRACT

Buses often get overcrowded, making travel uncomfortable and unsafe, and commuters have no way of knowing how crowded a bus is until it actually arrives at the stop. This project presents **Crowd Detection in a Bus**, a low-cost system that detects how crowded a bus is using photographs captured from inside the vehicle.

A camera mounted inside the bus captures a photo of the passenger area at regular intervals (every 10 seconds in the current implementation). A lightweight YOLOv8 nano object-detection model, designed to run on a Raspberry Pi Zero 2 W, detects and counts the people in each image. The passenger count is then mapped to a crowd level – **Low** (0–30 people), **Medium** (31–55 people) or **High** (56 or more) – using threshold values. The result is stored through a Flask-based REST backend and displayed on a live web dashboard, so that transit authorities and passengers can know the crowding status of a bus even before it reaches the stop.

YOLOv8 nano is fast, accurate and light enough for inexpensive embedded hardware, and the use of periodic photos instead of continuous video further reduces the processing load. The estimated hardware cost is approximately ₹2,700 per bus, which makes the solution affordable for an entire fleet and not just a pilot batch. The current prototype has been implemented and tested with person detection, counting, a live webcam feed and a Flask web dashboard. A multi-bus client–server version has also been implemented, in which the Raspberry Pi in each bus sends only its crowd level and route number to a central Flask server that loads Chennai bus routes from GTFS data and serves a mobile-friendly dashboard. The current prototype uses a pre-trained model without fine-tuning on in-bus images; deployment on the Raspberry Pi, testing with an actual bus camera view and threshold tuning form the next stage of work. The abstract of this work has been accepted for virtual presentation at the 13th International Multidisciplinary Conference on Current Research Trends (IMCCRT-2026) on 30 October 2026 (Abstract ID: IMCCRT-2026-2578).

Keywords: Crowd detection, passenger counting, YOLOv8, object detection, Raspberry Pi, OpenCV, Flask, public transport, edge computing.

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
Figure 3.1	Proposed system architecture for crowd detection in a bus	12
Figure 3.2	Data flow diagram of the crowd detection system	13
Figure 3.3	Hardware block diagram of the in-bus unit	15
Figure 4.1	Simplified YOLOv8 architecture used for person detection	18
Figure 4.2	Crowd levels and their people-count ranges	20
Figure 4.3	Flowchart of the crowd-level classification	20
Figure 5.1	Home screen of the Bus Crowd Tracker dashboard	25
Figure 5.2	Route search showing the matching Chennai routes	25
Figure 5.3	Bus detail view showing the crowd level and recent readings	26

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
Table 1.1	Functional overview of the system modules	4
Table 2.1	Summary of the reviewed studies	8
Table 3.1	Comparison of existing and proposed systems	11
Table 3.2	Data kept by the server for each bus	14
Table 3.3	Flask routes of the system	14
Table 3.4	Hardware requirements and estimated cost per bus	15
Table 3.5	Raspberry Pi Zero 2 W specification	16
Table 3.6	Software requirements	16
Table 4.1	Crowd-level thresholds	19
Table 5.1	Core functional evaluation checklist	24
Table 5.2	Planned evaluation metrics	25
Table 5.3	Hardware cost for different fleet sizes (₹2,700 per bus)	26
Table 5.4	Conference acceptance details	29
Table A.1	Files of the project	32

ABBREVIATIONS

ABBREVIATION	FULL FORM
AI	Artificial Intelligence
ML	Machine Learning
CNN	Convolutional Neural Network
YOLO	You Only Look Once
OpenCV	Open Source Computer Vision Library
API	Application Programming Interface
REST	Representational State Transfer
APC	Automatic Passenger Counter
IoT	Internet of Things
GPS	Global Positioning System
IR	Infrared
SQL	Structured Query Language
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
COCO	Common Objects in Context
IoU	Intersection over Union
NMS	Non-Maximum Suppression
MAE	Mean Absolute Error
FPS	Frames Per Second
RAM	Random Access Memory
OS	Operating System
SD	Secure Digital
WiFi	Wireless Fidelity
GTFS	General Transit Feed Specification
ROI	Region of Interest

CHAPTER 1 – INTRODUCTION

1.1 OVERVIEW OF THE PROJECT

Introduction

Public buses are the backbone of urban transportation, carrying millions of commuters every day. During peak hours, however, buses frequently become overcrowded, which reduces passenger comfort, affects safety and slows down boarding and alighting. Passengers waiting at a stop have no way of knowing how full an approaching bus is. By the time the bus arrives it is too late to plan around it – the passenger must either wait for the next bus or squeeze in anyway.

Modern computer-vision models can look at an image and recognise the people in it. Systems built on such models are far more informative than simple door sensors, because they can actually see how the passengers are spread through the vehicle. The difficulty is that many of these models are heavy and need powerful hardware, which makes them too costly to fit on every bus of a fleet.

This project, **Crowd Detection in a Bus – Real-Time Passenger Detection & Counting Using YOLO Object Detection**, addresses this problem. A camera placed inside the bus captures a photograph of the passenger area at regular intervals, and a lightweight YOLOv8 nano model counts the passengers in the image. The count is converted into a simple crowd level – Low, Medium or High – and pushed to a live dashboard that is visible to both commuters and transit authorities. The model is small enough to run on a low-cost Raspberry Pi Zero 2 W, and using photos instead of continuous video reduces the processing load further.

Problem Statement

The existing ways of estimating bus occupancy and informing passengers have several deficiencies:

- **Late information for commuters:** Passengers cannot judge how crowded a bus is until it reaches the stop, so they cannot plan their journey in advance.
- **No live fleet-level view for authorities:** Transit authorities lack a low-cost, live method of monitoring crowding across a fleet and therefore cannot adjust frequency or route allocation in time.
- **Inaccurate or expensive counting methods:** Manual counting and basic sensors are inaccurate, whereas dedicated Automatic Passenger Counter (APC) hardware is expensive to install on every bus.
- **Counts without context:** Infrared and WiFi based sensors report an occupancy number only; they give no visual confirmation and no information about how the crowd is distributed inside the bus.
- **Heavy models on the edge:** Accurate density-estimation networks and large detectors are too resource-intensive for low-cost in-vehicle hardware.

The main challenge is therefore the lack of an affordable, automatic and reasonably accurate system that measures crowding inside a bus and makes the result available to commuters and authorities in near real time.

Objectives of the Project

The objectives of this project are:

- To design a camera-based system that estimates the crowd level inside a bus automatically at regular intervals.
- To use a lightweight YOLOv8 nano model to detect and count passengers in captured images.
- To classify the passenger count into Low, Medium and High crowd levels using threshold values.
- To run the detection pipeline on low-cost edge hardware (Raspberry Pi Zero 2 W) with no cloud dependency for inference.
- To build a Flask-based backend and web dashboard that shows the live crowd status to commuters and transit authorities.
- To keep the total hardware cost per bus low enough (about ₹2,700) for fleet-wide deployment.

System Description

The system is organised as a pipeline of five cooperating modules. The output of each module is the input of the next: the **Image Capture & Trigger Module** obtains a photo of the passenger area at regular intervals; the **Person Detection Module** runs YOLOv8 nano and counts the passengers; the **Crowd Classification Module** converts the count to Low, Medium or High; the **Flask Backend & Dashboard Module** exposes the result through a REST API and a web page; and the **Data Storage & Live Update Module** keeps the latest level and a short history of each bus on the server so that the dashboard always shows the latest status.

Advantages of the System

- **Low cost:** Estimated hardware cost of about ₹2,700 per bus, a fraction of the cost of a dedicated APC.
- **Lightweight:** YOLOv8 nano runs on a Raspberry Pi Zero 2 W, and periodic photos instead of continuous video keep the processing load low.
- **Single camera:** No extra depth sensors, WiFi rigs or multi-camera setups are required.
- **Visual evidence:** Because the count comes from an image, it can be checked visually, unlike a bare sensor reading.
- **Commuter-facing output:** Passengers can see the crowd status before the bus arrives, in addition to the information available to authorities.
- **Scalable:** The same hardware and software can be replicated across the entire fleet.

Applications of the Project

- **City and intercity bus services:** Live crowd information for passengers waiting at stops.
- **Transit planning:** Route and frequency planning based on actual crowding data.
- **Safety management:** Identifying overloaded vehicles and reducing unsafe overcrowding.
- **Smart-city platforms:** Integration with passenger-information systems and mobile applications.

Expected Results

The project is expected to produce a working prototype that takes an image or a live camera feed as input, detects and counts the passengers with YOLOv8, classifies the crowd level as Low, Medium or High and shows the crowd level of each bus, together with its route, on a Flask web dashboard. It also lays the foundation for deployment on a Raspberry Pi Zero 2 W with an actual bus camera view.

Future Scope

Future versions can add GPS-based bus location, a mobile application with notifications, fine-tuning of the detector on in-bus images, stronger security of the REST API, historical crowd analytics for route planning, and optimisation of the model for faster inference on the Raspberry Pi.

1.2 SCOPE AND OBJECTIVE

Purpose of the Study

The purpose of this study is to design, implement and validate a low-cost, real-time system that measures crowding inside a bus and makes the information available to commuters and transit authorities. The project demonstrates that a simple camera setup combined with a lightweight deep-learning model can provide useful crowd information without expensive sensors or heavy computing hardware, and it examines how such a system can be made practical for an entire fleet rather than a pilot batch.

General Objective

The general objective is to develop an efficient, affordable and scalable AI-based system that automatically counts the passengers in a bus, classifies the crowd level and publishes the result on a live dashboard.

Specific Objectives

- **Environment and model setup:** To set up the Python environment and the YOLOv8 nano model for person detection.
- **Detection and counting:** To implement person detection and counting on images and on a live camera feed using OpenCV.

- **Crowd classification:** To implement the Low / Medium / High classification using the thresholds 0–30, 31–55 and 56 or more people.
- **Server and dashboard:** To develop a central Flask server and a mobile-friendly web dashboard (routes /, /api/report, /api/buses, /api/routes and /api/history) that show the live crowd level of each bus and let commuters search routes.
- **Edge deployment:** To deploy the model on a Raspberry Pi Zero 2 W and test it with an actual bus camera view.
- **Tuning:** To tune the crowd-level thresholds for the real bus environment.

Project Scope

A. Input scope. The system works on photographs of the passenger area taken at regular intervals (every 10 seconds in the current implementation). The current prototype also processes a live webcam feed for testing. Only the passenger area of a single bus is considered.

B. Functional scope. The system provides person detection with YOLOv8 nano, counting of detected persons, classification of the crowd into Low, Medium and High, and a live web dashboard. The multi-bus version additionally includes a client program that runs on the Raspberry Pi of each bus, a REST API protected by an API key, route information loaded from Chennai GTFS data, and in-memory storage of the latest level and the recent history of every bus.

C. Application scope. The system is aimed at city buses and intercity coaches and may be extended to stations and platforms. It benefits commuters directly and gives transit authorities live and historical crowd information.

System Functionality Overview

Table 1.1: Functional overview of the system modules

MODULE	MAIN INPUT	MAIN OUTPUT
Image Capture & Trigger	Capture timer (10 s), camera	Photo / frame of the passenger area
Person Detection (YOLOv8 nano)	Pre-processed frame	Person bounding boxes and passenger count
Crowd Classification	Passenger count	Crowd level: Low / Medium / High
Flask Backend & Dashboard	Bus ID, route and level reported by each bus	REST API responses and live web page
Data Storage & Live Update	Verified report	Latest level and recent history of each bus

Hardware and Software Specifications

The prototype is a Python application that runs on an ordinary Windows computer with a webcam, using Visual Studio Code for development. The target deployment uses a Raspberry

Pi Zero 2 W with a camera module and a stop sensor. Since the system uses a pre-trained detector for inference, no special training hardware is needed. The detailed hardware and software requirements are given in Sections 3.5 and 3.6.

Advantages of the Proposed System

- Commuters receive crowd information before the bus arrives.
- Authorities get a low-cost, live view of crowding across the fleet.
- Counting is based on what the camera actually sees, so results can be verified visually.
- The solution is cheap enough to be replicated across an entire fleet.

Issues and Limitations

At present the prototype uses a pre-trained detector without fine-tuning on in-bus images and has been tested on a laptop rather than on a bus. The deployment on the Raspberry Pi, threshold tuning and testing with an actual bus camera view are still to be completed. These issues are discussed further in Section 5.3.

Future Improvements

GPS integration, a commuter mobile application, fine-tuning on bus-interior data, API security, historical analytics and model optimisation for the Raspberry Pi are the planned improvements (Section 5.5).

Expected Outcome

The expected outcome is to show that a single camera, a lightweight YOLO model and a simple web dashboard can provide timely and understandable crowd information for buses at a cost suited to a whole fleet.

CHAPTER 2 – REVIEW OF THE LITERATURE

2.1 LITERATURE OVERVIEW

Estimating how many people are in a scene has been studied for decades. The earliest practical systems used simple physical sensors, such as infrared beams at the door or pressure mats, to count people passing a point. Computer vision then introduced hand-crafted detectors – the Viola–Jones cascade [13] and Histogram of Oriented Gradients with a support-vector machine [12] – which could find pedestrians in an image but were sensitive to lighting, occlusion and viewpoint. Deep learning changed this picture: region-based and single-shot detectors such as Faster R-CNN [14], SSD [15] and the YOLO family [7] learn features directly from data and detect people far more reliably, while density-estimation networks such as MCNN [16] and CSRNet [18] estimate the size of dense crowds without detecting each person.

In public transport the goal is to estimate the number of passengers in a vehicle so that operators and commuters can make better decisions. Recent work therefore combines lightweight networks with IoT hardware and dashboards. This review examines the related literature in order to identify the gap that the present project addresses. It serves the following purposes:

- To evaluate the sensor-based, classical computer-vision and deep-learning approaches used for people counting and crowd density estimation.
- To understand which model families are suitable for low-cost edge hardware.
- To examine existing IoT and public-transport crowd-monitoring systems.
- To identify the research gap and justify the design of the proposed system.

2.2 RESEARCH SUMMARY

The literature is summarised thematically below: sensor-based and classical approaches, deep-learning detectors, density estimation, lightweight models for edge devices, and IoT and public-transport crowd-monitoring systems.

Sensor-Based and Classical Approaches

Infrared and WiFi based counters give a count of people but no visual validation and no information on how the crowd is distributed in the vehicle; surveys of passenger-counting systems report that they are often combined with other sensors to improve accuracy. Classical computer-vision methods such as Viola–Jones [13] and HOG with an SVM [12] avoid extra sensors but struggle with changes in lighting, occlusion between passengers and perspective distortion – all of which are typical inside a bus.

Deep-Learning Detectors

Faster R-CNN [14] introduced a region-proposal network that made two-stage detection much faster, but it is still too heavy for small devices. SSD [15] and YOLO [7] are single-stage detectors that predict boxes and classes in one pass of the network and therefore run in real time. YOLO models are trained on large datasets such as MS COCO [19], in which "person" is one of

the classes, so a pre-trained YOLO model can detect passengers without any additional training. The YOLOv8 family from Ultralytics [8] provides several sizes, of which the nano model is the smallest and fastest.

Crowd Density Estimation

For very dense crowds, where individual people are too small or too occluded to be detected, density-estimation networks such as MCNN [16] and CSRNet [18] regress a density map whose integral is the crowd count. These models are accurate for large gatherings but are comparatively heavy, and they output a density map rather than the discrete person locations that are useful for visual verification. For the passenger numbers expected in a single bus, detection-based counting is sufficient.

Lightweight Models for Edge Devices

Ahmed et al. [1] built an IoT crowd-monitoring system using SSD with a MobileNetV2 backbone and transfer learning, confirming that a lightweight detector on constrained hardware is viable in real IoT deployments. MobileNetV2 [17] itself is an efficient architecture designed for mobile and embedded devices. Wang et al. [2] proposed a lightweight CNN based on a modified MobileNetV2 for crowd-density estimation, and Khan et al. [3] presented LCDnet, a lightweight real-time crowd-density model. Together these studies support the choice of a lightweight model family – here YOLOv8 nano – for the Raspberry Pi based edge device.

IoT and Public-Transport Crowd Monitoring

Zhang et al. [4] used video and GIS to map and classify crowd density, giving a precedent for converting raw counts into density classes, as the Low / Medium / High labels in this project do. Qaraqe et al. [5] presented PublicVision, a secure surveillance system using a Swin Transformer, which highlights the need for a secured backend and API; this shaped the authentication design of the Flask backend. Arun et al. [6] proposed YOLO-GHOST, an IoT-based smart crowd-monitoring system with a camera and a cloud dashboard, which is the closest existing system. This project differs by replacing continuous video and a cloud pipeline with periodic photo capture and on-device inference on a ₹2,700 Raspberry Pi per bus. Surveys of public-transport systems also show IoT, deep learning and WLAN-enabled mobile devices used together to estimate passenger counts and predict bus location, which reinforces a multi-sensor design that combines GPS with vision-based detection.

Summary of Related Work

Table 2.1: Summary of the reviewed studies

REF.	YEAR	TECHNIQUE	KEY CONTRIBUTION	GAP WITH RESPECT TO THIS PROJECT
[1]	2021	SSD + MobileNetV2 (transfer learning)	IoT crowd monitoring on constrained hardware	No live commuter-facing output
[2]	2022	Modified MobileNetV2 CNN	Lightweight crowd density estimation	Density output only; no passenger count per bus
[3]	2023	LCDnet	Lightweight real-time density model	No dashboard or deployment pipeline
[4]	2023	Video + GIS	Crowd density mapping and classification	Not designed for in-vehicle edge hardware
[5]	2024	Swin Transformer	Secure surveillance (PublicVision)	Heavy model; stresses need for secured API
[6]	2024	YOLO-GHOST	IoT smart crowd monitoring with cloud dashboard	Continuous video and cloud pipeline; higher cost
[12]	2005	HOG + SVM	Pedestrian detection	Sensitive to lighting and occlusion
[7]	2016	YOLO	Real-time single-stage detection	Original model; needs adaptation for in-bus use
[16]	2016	MCNN	Single-image crowd counting	Heavy; no person locations

Identified Gaps in Research

- **Non-visual sensors lack detail:** IR and WiFi based systems give occupancy counts only and provide no information on crowd distribution or visual confirmation.
- **Heavy models are unsuitable for edge devices:** Density-estimation CNNs and large YOLO variants are accurate but too resource-intensive for low-cost hardware.
- **No commuter-facing output:** Most existing work stops at detection or counting accuracy and does not provide a live crowd status for passengers.
- **High cost:** Depth sensors, multi-camera setups and dedicated APC hardware push up the cost per vehicle.

Literature Results Summary

The literature shows a clear move from physical sensors and hand-crafted detectors towards deep-learning detectors and, more recently, lightweight models suited to IoT hardware. YOLO-based detection and IoT dashboards have each been shown to work, but few systems combine a lightweight detector, on-device inference, a simple threshold-based crowd classification and a live commuter-facing dashboard on very cheap hardware. This project fills exactly that gap: a

lightweight YOLO-based person detector on a Raspberry Pi, threshold-based Low / Medium / High classification, and a live dashboard – balancing accuracy, cost and real-time feasibility.

CHAPTER 3 – SYSTEM DESIGN

3.1 EXISTING SYSTEM

Existing ways of measuring bus occupancy can be grouped into four categories.

Manual counting. Conductors or surveyors estimate the load by observation or by counting tickets. This is subjective, inaccurate during peak hours and cannot be shared with passengers in real time.

Door sensors (IR / WiFi). Infrared beams, pressure mats or WiFi probe counting estimate how many people entered and left. They give occupancy counts only, without visual validation or any detail about how the crowd is distributed inside the bus, and errors accumulate over a trip.

Classical computer-vision methods. Background subtraction and hand-crafted detectors avoid extra hardware but struggle with lighting changes, occlusion between standing passengers and perspective distortion.

Heavy deep-learning and dedicated hardware. Density-map CNNs, large YOLO variants, depth sensors, multi-camera rigs and dedicated Automatic Passenger Counters are accurate, but they need powerful hardware or expensive installations that are hard to justify for every bus of a fleet.

Limitations of the Existing System

- Commuters cannot see the crowd status before the bus reaches the stop.
- Counting methods are either inaccurate (manual, basic sensors) or expensive (dedicated APC hardware).
- Most research systems stop at detection or counting and do not provide a live, commuter-facing crowd status.
- Heavy models cannot run on low-cost in-vehicle hardware and rely on a cloud pipeline.
- No single system is both real-time and cheap enough to cover a whole fleet, not just a pilot batch.

3.2 PROPOSED SYSTEM

The proposed system is a low-cost, camera-based crowd detection system designed specifically for in-bus scenes. A single camera captures a photo of the passenger area at regular intervals. A YOLOv8 nano model running directly on a Raspberry Pi Zero 2 W detects and counts the passengers, and the count is converted into a Low, Medium or High crowd level. The result is stored through a Flask backend and displayed on a live dashboard for commuters and authorities.

Main Features

- **Lightweight detector:** YOLOv8 nano tuned for in-bus scenes.

- **On-device inference:** Runs directly on the Raspberry Pi Zero 2 W with no cloud dependency for inference.
- **Crowd classification:** The passenger count is converted into Low / Medium / High.
- **Live dashboard:** Status is pushed to a commuter-facing dashboard in addition to authorities.
- **Single camera:** No extra depth sensors, WiFi rigs or multi-camera setups are required.
- **Low cost:** Full hardware cost of about ₹2,700 per bus.

Why Periodic Photos Instead of Continuous Video

Processing continuous video needs far more computing power and bandwidth. Because the crowd level only needs to be known when the bus reaches a stop, a photo captured at regular intervals is enough. This reduces the processing load, makes it practical to run on the Raspberry Pi Zero 2 W and still delivers timely updates.

Advantages over the Existing System

Table 3.1: Comparison of existing and proposed systems

ASPECT	EXISTING SYSTEMS	PROPOSED SYSTEM
Counting method	Manual, IR / WiFi sensors or depth sensors	Camera image analysed by YOLOv8 nano
Visual validation	None for sensors; limited for classical vision	Annotated image with bounding boxes
Hardware	Dedicated APC or heavy computing hardware	Raspberry Pi Zero 2 W with a single camera
Cost per bus	High for APC / multi-camera setups	About ₹2,700
Processing	Continuous video or cloud pipeline	Periodic photos processed on-device
Output	Occupancy count for operators	Low / Medium / High status on a live dashboard
Audience	Operators only	Commuters and transit authorities

3.3 SYSTEM ARCHITECTURE

Architectural Overview

The architecture of the system is shown in Figure 3.1. It consists of a capture layer inside the bus, a processing layer that performs person detection and counting, a storage layer that keeps the recorded frames and results, and an output layer that presents the crowd status. Crowd monitoring starts with the in-bus camera, which captures images at regular intervals. The images are converted to frames and sent to the server, where the recorded frames are stored in a database. The stored frames are used for training and testing of the YOLOv8 model, and the server performs person detection and person counting through the YOLOv8 pipeline.

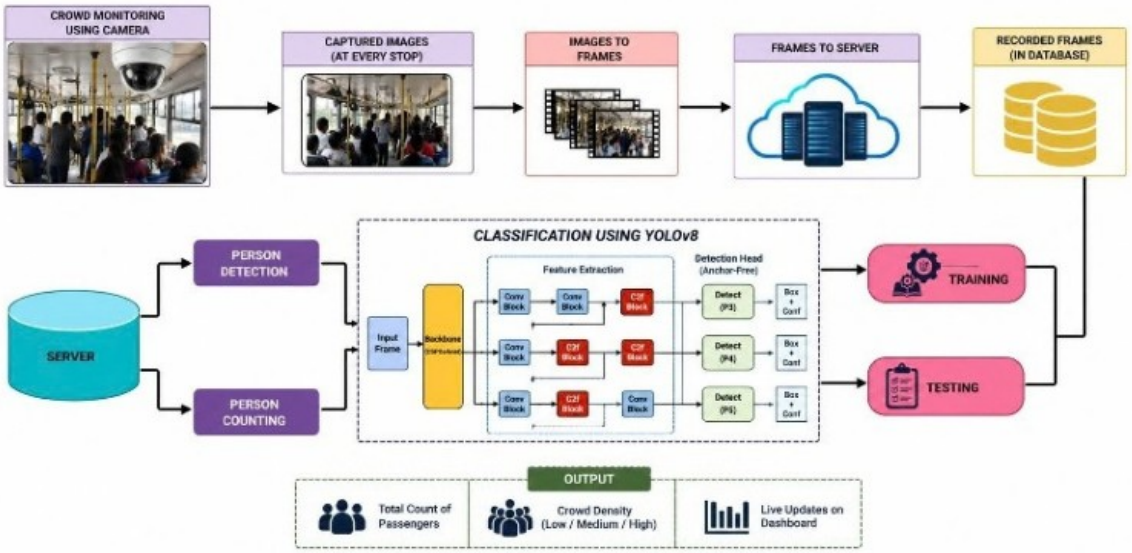


Figure 3.1: Proposed system architecture for crowd detection in a bus

Step 1: Capture

The in-bus camera monitors the passenger area. Every 10 seconds the program running on the Raspberry Pi takes a photo of the passenger area. An IR / ultrasonic stop sensor can be added to trigger an additional capture at each stop. In the initial prototype, the laptop webcam provides frames continuously for testing.

Step 2: Images to Frames and Frames to Server

The captured image is converted to a frame and pre-processed with OpenCV. The frame is passed to the detection module on the Raspberry Pi. The recorded frames and results are stored in the database, so that they can later be used for training, testing and tuning of the thresholds.

Step 3: Classification using YOLOv8

YOLOv8 consists of a backbone that extracts features from the input frame, a neck that merges features of different scales and an anchor-free detection head that outputs a bounding box and a confidence score for each detected object. Person detection locates every passenger and person counting returns their number. The architecture of YOLOv8 is explained in Section 4.1.

Step 4: Output

The system produces three outputs: the **total count of passengers**, the **crowd density** (Low, Medium or High) and **live updates on the dashboard** for commuters and authorities.

3.4 DATA FLOW / WORKFLOW

Figure 3.2 shows how data moves through the system from the moment the camera takes a photo until the commuter sees the result on the dashboard.

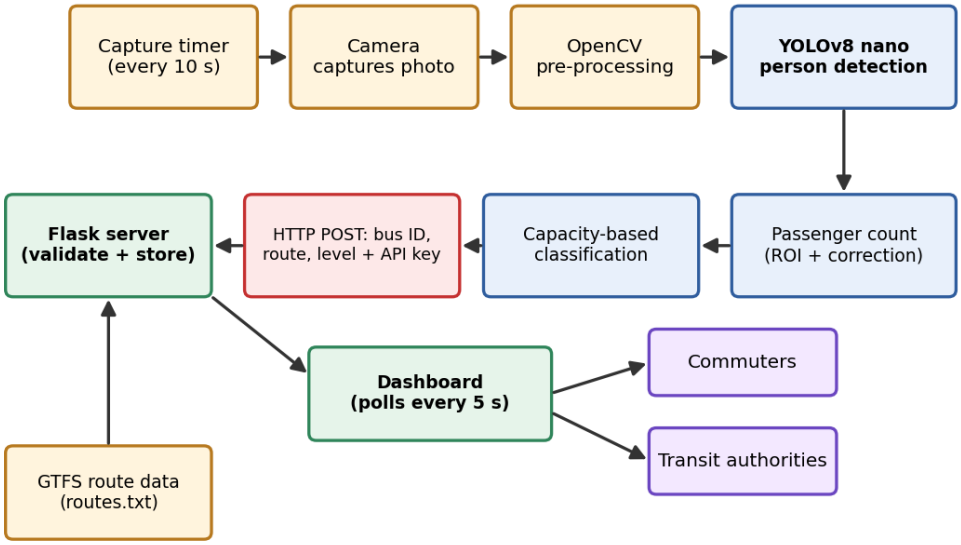


Figure 3.2: Data flow diagram of the crowd detection system

Workflow Steps

1. **Capture:** Every CAPTURE_INTERVAL (10 seconds in the current implementation), the program bus_client.py running on the Raspberry Pi takes a photo of the passenger area with the Pi camera module or a USB webcam. A stop sensor can additionally trigger a capture at each stop.
2. **Pre-processing:** The frame is captured at 640 × 480 pixels; the first few frames are discarded to let the camera settle.
3. **Detection:** YOLOv8 nano detects persons in the frame; only detections with a confidence of at least 0.4 are kept.
4. **Counting:** The number of persons (optionally only those inside a region of interest) is counted.
5. **Classification:** The count is mapped to LOW, MEDIUM or HIGH using thresholds derived from the seating and standing capacity of the bus.
6. **Reporting:** The bus sends its bus ID, route number and crowd level – not the photo and not the count – to the server as an HTTP POST request to /api/report, together with an API key.
7. **Storage:** The Flask server validates the report, stores the latest level of the bus and appends it to a short history of the last 60 readings.
8. **Live update:** The dashboard polls the server every 5 seconds and shows each reporting bus with its route, destination and crowd level. A bus that has not reported for 120 seconds is shown as OFFLINE and is removed from the list after 600 seconds.

Data Stored by the Server

The server keeps the data in memory, so no database installation is needed on the server. For every bus the information in Table 3.2 is stored; in addition, the route table loaded from the GTFS file maps each route number (for example 21G) to its name. A SQL database (SQLite or MySQL) is planned for long-term storage of the readings.

Table 3.2: Data kept by the server for each bus

FIELD	TYPE	DESCRIPTION
bus_id	Text (1–30 characters)	Unique identifier of the bus, for example its registration number; letters, digits, space, - or _
route	Text	Route number the bus is running, converted to upper case without spaces (for example 21G)
level	LOW / MEDIUM / HIGH	Latest crowd level reported by the bus
time	Text (HH:MM:SS)	Time of the latest report, shown on the dashboard
ts	Number	Timestamp of the latest report, used to detect OFFLINE buses
history	List of 60 entries	Most recent readings, each with a time and a level

Interface Design

The routes of the Flask server (server.py) and of the initial prototype application (app.py) are given in Table 3.3.

Table 3.3: Flask routes of the system

ROUTE	METHOD	PURPOSE
/	GET	Serves the dashboard page (index.html)
/api/report	POST	Called by each bus with {bus_id, route, level}; requires the X-API-Key header; validates and stores the reading
/api/buses	GET	Lists all reporting buses with route, destination name, crowd level (or OFFLINE) and age of the latest reading
/api/routes?q=	GET	Searches Chennai bus routes by number or name and returns up to 15 matches with the number of live buses on each
/api/history/<bus_id>	GET	Returns the recent readings (up to 60) of a bus
/video_feed, /status (prototype app.py)	GET	Live annotated camera feed and latest people count and crowd level in the initial single-camera prototype

The server rejects a report with an invalid API key (HTTP 401), an invalid bus ID, a missing route or a level other than LOW, MEDIUM or HIGH (HTTP 400), and a route number that does not exist in the Chennai GTFS data (HTTP 404).

3.5 HARDWARE REQUIREMENTS

The hardware needed for one bus is listed in Table 3.4 and its connections are shown in Figure 3.3. The estimated total cost is ₹2,700 per bus, a fraction of the cost of a dedicated Automatic Passenger Counter.

Table 3.4: Hardware requirements and estimated cost per bus

S.NO	COMPONENT	DESCRIPTION	COST (₹)
1	Camera Module	OV2640 / low-cost USB camera for photo capture	350
2	Processing Unit	Raspberry Pi Zero 2 W, runs YOLO inference on-device	1,500
3	Stop Sensor (optional)	IR / ultrasonic sensor to trigger an extra photo capture at each stop	150
4	MicroSD Card	16 GB Class 10, stores OS, model weights and photos	300
5	Power Supply	12V → 5V buck converter, powered off the bus line	150
6	Enclosure	3D-printed / acrylic housing to protect the camera and Pi	250
	TOTAL		**2,700**

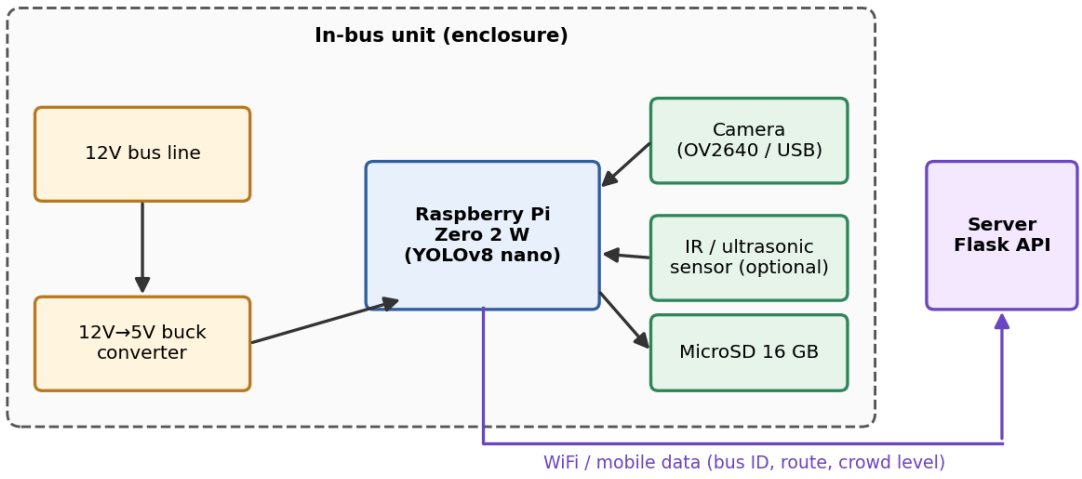


Figure 3.3: Hardware block diagram of the in-bus unit

Processing Unit: Raspberry Pi Zero 2 W

The Raspberry Pi Zero 2 W is a very small, low-power single-board computer. Its main characteristics are listed in Table 3.5. Its limited memory and processor speed are the reason why the smallest YOLOv8 model and periodic photo capture are used.

Table 3.5: Raspberry Pi Zero 2 W specification

FEATURE	SPECIFICATION
Processor	Quad-core 64-bit ARM Cortex-A53 at 1 GHz
Memory	512 MB LPDDR2 SDRAM
Connectivity	2.4 GHz 802.11 b/g/n WiFi, Bluetooth 4.2
Camera interface	CSI-2 camera connector (USB camera via adapter)
Storage	MicroSD card slot
Power	5 V supply (from the 12V → 5V buck converter)

Prototype Development Machine

The present prototype was developed and tested on a Windows laptop with a built-in webcam, using Visual Studio Code and Python. A modern processor with at least 8 GB of RAM is recommended for smooth live-feed processing on a computer; a GPU is optional.

3.6 SOFTWARE REQUIREMENTS

Table 3.6: Software requirements

COMPONENT	SPECIFICATION	PURPOSE
Detection Model	YOLOv5 / YOLOv8 nano (Ultralytics)	Real-time object (person) detection
Programming Language	Python	Model inference and system logic
Computer Vision Library	OpenCV	Photo capture and pre-processing
Backend Framework	Flask	REST API serving detection results
Frontend	HTML, CSS, JavaScript	Dashboard page
Data storage	In-memory (current); MySQL / SQLite (planned)	Latest level and recent history of each bus
Route data	GTFS (chennai-unified-gtfs.zip)	Chennai bus route numbers and names
HTTP client	Python requests	Bus client sends the level to the server

COMPONENT	SPECIFICATION	PURPOSE
Operating System	Raspberry Pi OS (device); Windows (prototype)	On-device deployment and development
Editor	Visual Studio Code	Development environment
Browser	Google Chrome / Mozilla Firefox	Viewing the dashboard

CHAPTER 4 – IMPLEMENTATION

4.1 METHODOLOGY

The project follows a modular development approach carried out over a 16-week schedule (July to October). The activities were problem identification and planning, literature survey, requirements and system design, YOLOv8 setup and dataset preparation, person-detection implementation, live-camera integration, Flask web dashboard development, testing and crowd classification, and finally results, documentation and review.

Object Detection with YOLOv8

YOLO ("You Only Look Once") [7] treats object detection as a single regression problem. Instead of first proposing regions and then classifying them, the network looks at the whole image once and predicts bounding boxes and class probabilities directly, which makes it fast enough for real-time use. YOLOv8 [8] is a recent member of the family and is available in several sizes (nano, small, medium, large and extra-large); the nano model has the fewest parameters and is therefore the best fit for the Raspberry Pi Zero 2 W.

The architecture of YOLOv8, shown in Figure 4.1, has three parts. The **backbone** is a convolutional network built from Conv and C2f blocks, ending in an SPPF block, which extracts features at several scales. The **neck** (a PAN-FPN structure) merges the features by upsampling and down-sampling so that both small and large objects are represented. The **detection head** is anchor-free and decoupled: it predicts the box coordinates and the class confidence separately at three scales (P3, P4 and P5). Finally, non-maximum suppression (NMS) removes duplicate boxes.

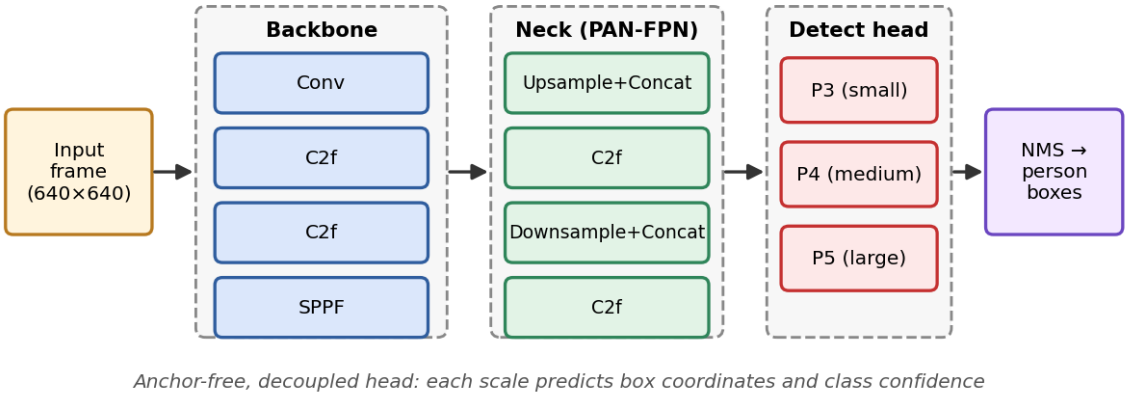


Figure 4.1: Simplified YOLOv8 architecture used for person detection

Person Counting

The pre-trained YOLOv8 nano model has been trained on the MS COCO dataset [19], in which "person" is class 0. For every frame the model is asked to report only this class. If a frame

produces boxes B_1 … B_k with confidence scores c_1 … c_k, the passenger count N is the number of boxes whose confidence exceeds the confidence threshold τ:

$$N = \Sigma \; 1 \; [\; class(B_i) = person \;\; and \;\; c_i \geq \tau \;] \qquad (1)$$

When two boxes cover the same person, NMS keeps the one with the higher confidence and discards the other if their overlap, measured by the Intersection over Union (IoU), is larger than a set limit:

$$IoU(A, B) = | A \cap B | / | A \cup B | \qquad (2)$$

where A and B are two bounding boxes, |A ∩ B| is the area of their intersection and |A ∪ B| is the area of their union. An IoU of 0 means that the boxes do not overlap and 1 means that they coincide.

In the implementation the confidence threshold is τ = 0.4. Only persons whose box centre lies inside an optional region of interest (ROI) are counted, so that areas outside the bus can be ignored. Passengers hidden behind others may be missed, so the count can be multiplied by a correction factor f (default 1.0): N_est = round(f × N).

Crowd-Level Classification

The passenger count is converted into a crowd level using thresholds that depend on the capacity of the bus (Equations 3 and 3a, Table 4.1 and Figure 4.2):

$$Level(N) = LOW \text{ if } N \leq L; \;\; MEDIUM \text{ if } L < N \leq M; \;\; HIGH \text{ if } N > M \qquad (3)$$

$$L = round(0.75 \times S), \;\; M = S + round(0.5 \times T) \qquad (3a)$$

where S is the seating capacity and T the standing capacity of the bus. The low limit L is 75 % of the seats, and the medium limit M is the number of seats plus half of the standing places. For the bus considered here (S = 40, T = 30) the limits are L = 30 and M = 55.

Table 4.1: Crowd-level thresholds

CROWD LEVEL	PEOPLE COUNT	MEANING
LOW	0 – 30	Plenty of seats (up to 75 % of the seats taken)
MEDIUM	31 – 55	Seats full, some passengers standing
HIGH	56 or more	Packed; consider waiting for the next bus

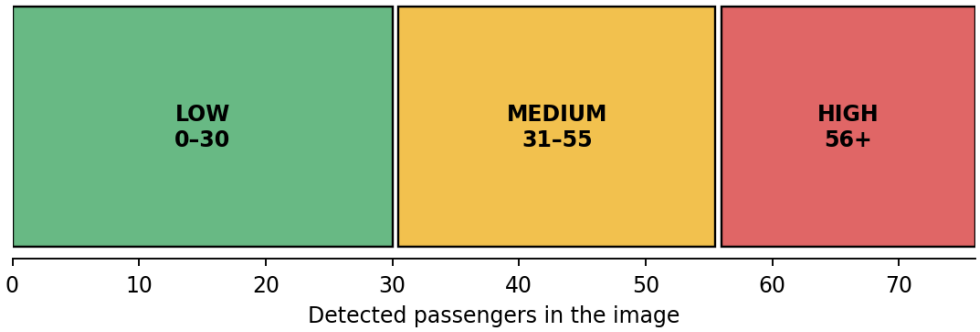


Figure 4.2: Crowd levels and their people-count ranges

For example, a detected count of 40 is classified as MEDIUM. The decision logic is shown as a flowchart in Figure 4.3. The limits follow automatically from the capacity values set for each bus and can be tuned for the real bus environment during on-vehicle testing.

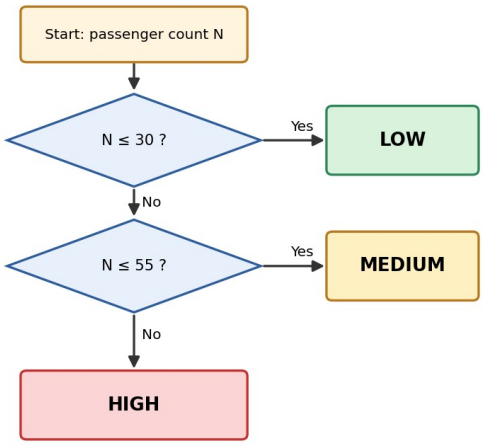


Figure 4.3: Flowchart of the crowd-level classification

Privacy by Design

Because the system works with images of passengers, privacy is considered from the start. The system does not identify people – it only counts them. The photo is processed on the Raspberry Pi and is never sent to the server; only the bus ID, the route number and the crowd level (LOW, MEDIUM or HIGH) are transmitted. The reporting API is protected by an API key so that only authorised bus units can post readings.

4.2 IMPLEMENTATION

The implementation consists of three programs (the listings of bus_client.py and server.py are given in Appendix A): **bus_client.py**, which runs on the Raspberry Pi inside each bus; **server.py**, the central Flask server that runs once; and **templates/index.html**, the commuter dashboard. The initial single-camera prototype (test_yolo.py, count_people_yolo.py and app.py) was developed first on a laptop with a webcam. The screens of the final dashboard are shown in Figures 5.1 to 5.3. The final data flow is **Camera → OpenCV → YOLOv8 → Count → Crowd Level → HTTP POST → Flask server → Dashboard**.

The project folder (YOLO) contains the files listed in Table A.1 of Appendix A, including the model weights yolov8n.pt and the Chennai route data chennai-unified-gtfs.zip.

4.2.1 Image Capture & Trigger Module

This module obtains the images that are analysed by the model. It is implemented in the function capture_photo() of bus_client.py.

Technologies used: OpenCV, Python, Picamera2 (Pi Camera Module) or a USB webcam.

Implementation steps:

1. **Camera selection:** If USE_PICAMERA2 is True, the Pi Camera Module is used through Picamera2; otherwise a USB webcam is opened with cv2.VideoCapture(CAMERA_INDEX).
2. **Settings:** The frame size is set to 640 × 480 pixels.
3. **Warm-up:** Several frames (WARMUP_FRAMES = 5) are read and only the last one is kept, so that the exposure of the camera settles.
4. **Release:** The camera is released after every photo, so that it is not kept busy between captures.
5. **Timing:** A main loop repeats the cycle every CAPTURE_INTERVAL seconds (10 s). A stop sensor can be added to trigger a capture at each stop.

Output: A frame (image) of the passenger area ready for detection.

4.2.2 Person Detection Module (YOLOv8)

This is the core AI component of the system, implemented in the function count_people().

Algorithm used: YOLOv8 nano (yolov8n.pt) from Ultralytics.

Implementation steps:

1. **Model loading:** The pre-trained weights are loaded once at start-up: model = YOLO("yolov8n.pt").
2. **Inference:** The frame is passed to the model with a confidence threshold of 0.4 (CONF_THRESHOLD), restricted to the person class (class 0).
3. **Region of interest:** If a region-of-interest polygon (ROI_POLYGON) is set, only the persons whose box centre lies inside it are counted. This allows areas outside the bus, such as windows or doors, to be ignored.
4. **Counting:** The number of accepted person boxes is the passenger count (Equation 1).
5. **Correction:** The count is multiplied by DETECTION_CORRECTION (default 1.0) and rounded. A value above 1 can be used if the camera view misses some passengers.

Output: The estimated number of passengers in the photo.

4.2.3 Crowd Classification Module

This module, implemented in classify_level(), converts the count into a crowd level using the threshold logic of Equations 3 and 3a. The limits are not typed in by hand but are computed

from the seating capacity (SEATING_CAPACITY = 40) and the standing capacity (STANDING_CAPACITY = 30) of the bus, which gives LOW_MAX = 30 and MEDIUM_MAX = 55. A count up to 30 gives **LOW**, a count from 31 to 55 gives **MEDIUM**, and 56 or more gives **HIGH**. The capacities can be set separately for each type of bus. **Output:** a text label (LOW / MEDIUM / HIGH), which is the only result that leaves the bus.

4.2.4 Flask Backend & Dashboard Module

This module provides the central server and the interface through which commuters see the results.

Technologies used: Flask, Python, HTML, CSS, JavaScript, GTFS route data.

Key features:

1. **Route data from GTFS:** At start-up server.py reads routes.txt from chennai-unified-gtfs.zip and keeps the bus routes (metro and train lines are skipped), so that route numbers and names need not be typed by hand. If the file is missing, the server runs in open mode and accepts any route number.
2. **Reporting API:** The route /api/report accepts the reports of the buses. A report is accepted only with the correct API key and with a valid bus ID, an existing route and a level of LOW, MEDIUM or HIGH. A bus is registered automatically the first time it reports.
3. **Query API:** The routes /api/buses, /api/routes and /api/history/<bus_id> provide the list of buses, the route search and the recent readings of a bus (Table 3.3).
4. **Dashboard page:** The page Bus Crowd Tracker (index.html) is designed for the screen of a mobile phone. It offers a search box for routes, filter buttons (All buses, Seats free, Filling up, Crowded), a summary of how many buses have seats free, and a card for each bus with its route number, destination, level and the time since the last report (Figures 5.1 and 5.2).
5. **Bus detail view:** Selecting a bus shows its crowd level with a Low–Medium–High scale, a short advice for the passenger, and a strip of the recent readings (Figure 5.3).
6. **Live refresh:** The page polls /api/buses every 5 seconds and refreshes the history of the open bus every third poll; if the server cannot be reached, the buses are shown as having no reading.

Output: A web page that shows the crowd level of every reporting bus and lets commuters search any Chennai route to see whether a bus on it is reporting.

4.2.5 Data Storage & Live Update Module

The server keeps the latest report of each bus and its last 60 readings in memory, protected by a lock because the server runs in threaded mode (Table 3.2). A bus that has not reported for 120 seconds (OFFLINE_AFTER) is shown as OFFLINE, and a bus that has been silent for 600 seconds (REMOVE_AFTER) is removed from the list. Only a small record is kept for each bus,

so memory and bandwidth requirements are minimal. Persistent storage in a SQL database is planned for the next stage.

4.3 INTEGRATION AND DEPLOYMENT

Integration Summary

1. bus_client.py captures a photo every 10 seconds.
2. YOLOv8 nano detects the persons and returns the count.
3. The classification logic converts the count into LOW / MEDIUM / HIGH.
4. The bus sends its ID, route number and level to /api/report on the server, with the API key.
5. server.py validates the report, stores it and makes it available through /api/buses and /api/history.
6. The dashboard polls the server and shows the crowd level of every reporting bus.

Deployment Plan

The system is deployed in two parts: one copy of the client on each bus and one copy of the server.

1. **Server:** Install Flask, place the GTFS zip in the server folder, set the API key in the environment variable BUS_API_KEY and start the server with python3 server.py. The server can run on a laptop, a college server or a cloud virtual machine.
2. **Bus unit:** Install Raspberry Pi OS and the packages ultralytics, opencv-python and requests (plus picamera2 for the Pi Camera Module) on the Raspberry Pi Zero 2 W.
3. **Configuration:** In bus_client.py set BUS_ID (unique for each vehicle), ROUTE_NO (the route the bus is running), SERVER_URL (address of the server), API_KEY (same as on the server), the camera type and the seating and standing capacity of the bus.
4. **Hardware:** Connect the camera, power the unit through the 12V → 5V buck converter and fit it in its enclosure.
5. **Test and tuning:** Test with the actual bus camera view and tune the region of interest, the correction factor and the crowd-level thresholds.

This modular implementation keeps the heavy work (detection) on the bus and sends only a tiny message to the server, which makes it easy to scale the system from one bus to a whole fleet.

CHAPTER 5 – RESULTS, CONCLUSION AND FUTURE WORK

5.1 RESULTS AND EVALUATION

The prototype was run on a laptop and tested with a webcam. The evaluation at this stage checks whether each module produces a useful output and whether the outputs are combined correctly and shown on the dashboard. The implementation results are summarised in Table 5.1.

Table 5.1: Core functional evaluation checklist

TEST	SCENARIO	EXPECTED OUTCOME	OBSERVED RESULT / STATUS
T01	Model loading	YOLOv8 nano weights load and run	Model loaded and tested successfully
T02	Person detection and counting	Persons in an image are boxed and counted	Implemented; persons detected and counted
T03	Live webcam feed	Frames are processed continuously	Live feed successfully tested
T04	Crowd-level classification	Count mapped to Low / Medium / High	Implemented
T05	Flask dashboard	Live crowd status of the bus shown on a web page	Implemented (initial prototype page and final index.html dashboard)
T06	Raspberry Pi deployment	Model runs on the Pi Zero 2 W	Planned (next stage)
T07	Actual bus camera view	Counting works on in-bus images	Planned (next stage)
T08	Threshold tuning	Thresholds suited to the bus environment	Planned (next stage)
T09	Multi-bus client–server pipeline	Level sent by bus_client.py appears on the dashboard	Implemented (bus_client.py, server.py, index.html); test with real buses planned

Figures 5.1 to 5.3 show the final Bus Crowd Tracker dashboard (index.html) served by the Flask server (server.py) and opened in a web browser at 127.0.0.1:5000. At the time of the test one bus (BUS001) was reporting on Route 21G.

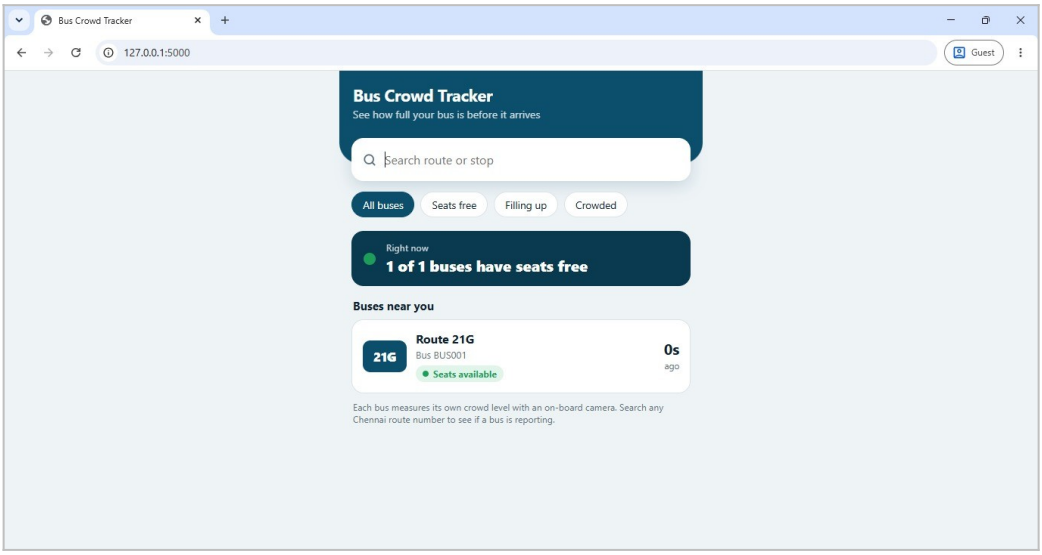


Figure 5.1: Home screen of the Bus Crowd Tracker dashboard

Figure 5.1 shows the home screen. It has a search box for a route or stop, the filter buttons (All buses, Seats free, Filling up, Crowded), a banner showing how many buses have seats free, and a card for each reporting bus with its route number, bus ID, crowd status and the time since its last report.

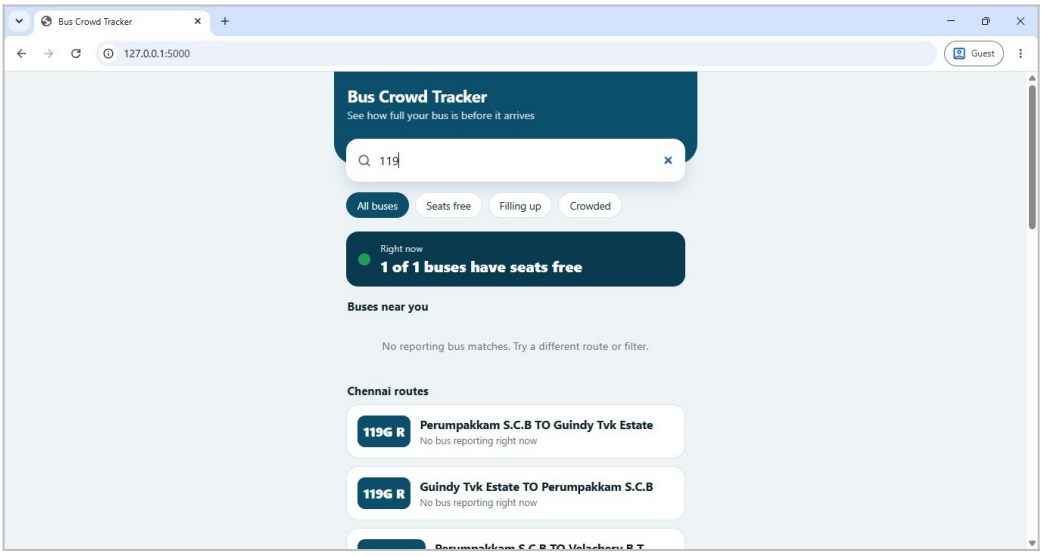


Figure 5.2: Route search showing the matching Chennai routes

Figure 5.2 shows the route search. When no bus is reporting on the route that was typed (here 119), the page says so and lists the matching Chennai routes from the GTFS data, each marked “No bus reporting right now”.

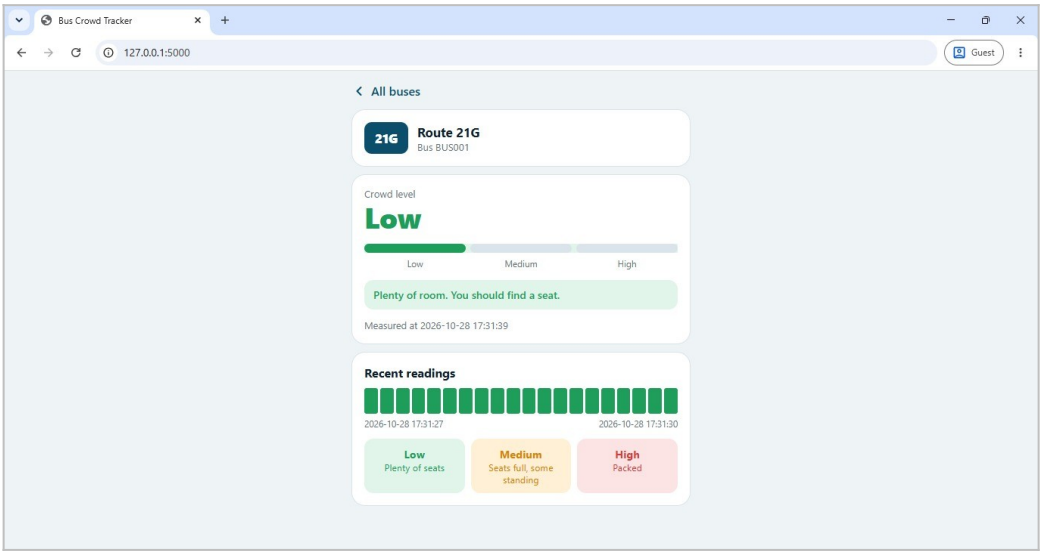


Figure 5.3: Bus detail view showing the crowd level and recent readings

Figure 5.3 shows the detail view of Route 21G. It displays the current crowd level (Low) on the Low–Medium–High scale, a short advice for the passenger, the time of measurement, a strip of the recent readings and a legend that explains each level.

Evaluation Plan for the Next Stage

No measured accuracy or speed value is claimed in this report, because the system has not yet been evaluated on in-bus images. In the next stage the following measurements will be made on a set of real bus photographs whose true passenger counts are noted manually (Table 5.2).

Table 5.2: Planned evaluation metrics

METRIC	DEFINITION	PURPOSE
Mean absolute error (MAE)	Average of |detected − true| passenger counts	How close the counts are to reality
Crowd-level accuracy	Correct levels ÷ total images × 100	How often Low / Medium / High is right
Inference time per image	Time taken by the Pi to process one photo	Real-time feasibility on the Pi
Update delay	Time from capture to dashboard update	Timeliness for commuters
Robustness	Behaviour under low light, occlusion and crowding	Reliability in real conditions

$$MAE = (1 / M) \Sigma \mid N_i(detected) - N_i(true) \mid \quad (4)$$

$$Accuracy = (Correct\ levels / M) \times 100\ \% \quad (5)$$

where M is the number of test images. Based on these measurements, the thresholds and, if necessary, the confidence threshold τ will be adjusted.

Cost Analysis

The cost of the system grows linearly with the size of the fleet because each bus needs one unit costing about ₹2,700 (Table 3.4). Table 5.3 shows the hardware cost for fleets of different sizes.

Table 5.3: Hardware cost for different fleet sizes (₹2,700 per bus)

NUMBER OF BUSES	TOTAL HARDWARE COST (₹)
1	2,700
50	1,35,000
100	2,70,000
500	13,50,000

5.2 DISCUSSION

The main achievement of the project is a complete, working path from camera to dashboard: an image is captured, the passengers are detected and counted by YOLOv8 nano, the count is classified as Low, Medium or High, and the result is shown on a web page. The pipeline was kept deliberately simple so that every stage can run on very modest hardware.

The decision to use periodic photos rather than continuous video follows from how the information is used: a commuter needs to know how crowded a bus is when it leaves a stop, not frame by frame. This makes on-device inference on a Raspberry Pi Zero 2 W realistic and keeps the cost per bus low. The threshold-based classification is easy to understand for commuters, but the thresholds themselves are a design choice and must be validated against real buses.

It should be noted that the detected count is an estimate. In a crowded bus some passengers are partly or fully hidden by others, and a detector trained on general scenes may miss them. The classification into three broad levels is more tolerant of small counting errors than an exact count would be, but the effectiveness of the system can only be stated after the evaluation of Section 5.1 has been completed on real bus images.

5.3 CHALLENGES AND LIMITATIONS

- **Occlusion:** Passengers standing close together partly hide one another, which can lead to under-counting in crowded conditions.
- **Lighting and camera position:** Poor lighting, glare through windows and an unsuitable camera angle can reduce detection quality.
- **Snapshot, not continuous:** Photos are captured at intervals, so the reported level is a snapshot at the stop rather than a continuous measurement.
- **Limited hardware:** The Raspberry Pi Zero 2 W has a slow processor and only 512 MB of memory, so inference takes longer than on a PC.

- **General-purpose model:** The prototype uses a pre-trained model without fine-tuning on in-bus images.
- **Threshold validity:** The Low / Medium / High thresholds have not yet been tuned on a real bus.
- **Connectivity:** The bus unit needs a WiFi or mobile-data link to send readings to the server.
- **In-memory storage:** The server keeps its data in memory, so the readings are lost when the server restarts and no long-term history is available.
- **Basic security:** All buses share one API key and the prototype communicates over plain HTTP.
- **Privacy and security:** Images of passengers are sensitive. Images should be processed on the device and deleted, and the API must be authenticated.

5.4 CONCLUSION

Overcrowding on public buses affects passenger comfort, safety and transit efficiency. This project built a real-time crowd detection system using a camera and the YOLOv8 nano model, designed for a low-cost Raspberry Pi Zero 2 W. Photos captured at regular intervals are processed on-device to count passengers and classify the crowd level as Low, Medium or High, and the results are pushed through a Flask backend to a live dashboard for commuters and authorities.

Achievements

- The YOLOv8 model was successfully loaded and tested.
- Person detection and counting were implemented.
- The live webcam feed was successfully tested.
- The crowd-level classification (Low / Medium / High) was defined and implemented.
- A multi-bus client–server version was implemented: bus_client.py sends the crowd level of each bus to a central Flask server (server.py), which loads Chennai bus routes from GTFS data and serves a mobile-friendly dashboard (index.html) with route search and per-bus history.

Significance of the System

Periodic photo capture instead of continuous video keeps the processing light for embedded hardware while still delivering timely updates. At an estimated cost of about ₹2,700 per bus, the system is affordable and practical across an entire fleet, not just a pilot batch. A simple camera setup with a lightweight deep-learning model can meaningfully improve route planning and the passenger experience, and provides a scalable foundation for smarter, data-driven public transportation.

5.5 FUTURE WORK

Next Implementation Stage

- Connect a camera to the Raspberry Pi Zero 2 W.
- Deploy the YOLOv8 model on the Raspberry Pi.
- Test the system with an actual bus camera view.
- Tune the crowd-level thresholds for the bus environment.
- Deploy the live dashboard for monitoring.

Further Enhancements

- **Fine-tuning on in-bus data:** Train or fine-tune the model on a dataset of bus interiors, including head-level detection, to improve accuracy under occlusion.
- **GPS integration:** Combine crowd data with the bus location so that commuters can see both the position and the crowding of an approaching bus.
- **Mobile application:** Provide passengers with a mobile app and notifications.
- **Analytics:** Use the stored records for route planning, demand forecasting and peak-hour analysis.
- **Persistent storage:** Store the readings in a SQL database (SQLite or MySQL) for long-term history and analysis.
- **Stronger security:** Use HTTPS and a separate API key for every bus instead of one shared key.
- **Stop-based capture:** Add the IR / ultrasonic stop sensor so that an extra photo is taken at every stop.
- **Model optimisation:** Explore quantisation or conversion of the model (for example to NCNN or ONNX) to speed up inference on the Raspberry Pi.

5.6 PUBLICATION OUTCOME

The work presented in this report has also been accepted for presentation at an international conference. The abstract titled "Crowd Detection in a Bus" was accepted for the 13th International Multidisciplinary Conference on Current Research Trends (IMCCRT-2026), to be presented in the online (virtual) mode on 30 October 2026. The acceptance letter, issued by the IMCCRT-2026 Secretariat on 12 October 2026, is reproduced in Appendix B and its main details are summarised in Table 5.4.

Table 5.4: Conference acceptance details

ITEM	DETAILS
Paper / abstract title	Crowd Detection in a Bus
Conference	13th International Multidisciplinary Conference on Current Research Trends (IMCCRT-2026)
Abstract ID	IMCCRT-2026-2578
Date of acceptance letter	12 October 2026
Presentation date and mode	30 October 2026, online platform (virtual presentation)
Publication	Accepted abstracts are added to the conference proceedings with an ISBN number
Next step	Submission of the full paper to the conference secretariat for review

This acceptance shows that the idea of a low-cost, camera-based crowd-detection system for buses is of interest to the research community, and it gives the team an opportunity to present the work and receive feedback, which will be used in the next stage of the project.

REFERENCES

[1] I. Ahmed et al., "IoT-based crowd monitoring using SSD and MobileNetV2 with transfer learning," 2021.

[2] S. Wang et al., "Lightweight CNN (modified MobileNetV2) for crowd density estimation," 2022.

[3] M. A. Khan et al., "LCDnet: A lightweight real-time crowd density estimation model," 2023.

[4] X. Zhang et al., "Crowd density mapping and classification using video and GIS," 2023.

[5] M. Qaraqe et al., "PublicVision: A secure smart surveillance system for crowd behavior recognition using a Swin Transformer," 2024.

[6] D. R. Arun et al., "YOLO-GHOST: IoT-based smart crowd monitoring using camera and cloud dashboard," 2024.

[7] J. Redmon, S. Divvala, R. Girshick and A. Farhadi, "You Only Look Once: Unified, real-time object detection," in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2016.

[8] G. Jocher, A. Chaurasia and J. Qiu, "Ultralytics YOLOv8," Ultralytics, 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[9] G. Bradski, "The OpenCV Library," Dr. Dobb's Journal of Software Tools, 2000.

[10] Flask Documentation, Pallets Projects. [Online]. Available: https://flask.palletsprojects.com

[11] Raspberry Pi Zero 2 W documentation, Raspberry Pi Ltd. [Online]. Available: https://www.raspberrypi.com/documentation/

[12] N. Dalal and B. Triggs, "Histograms of oriented gradients for human detection," in Proc. IEEE CVPR, 2005.

[13] P. Viola and M. Jones, "Rapid object detection using a boosted cascade of simple features," in Proc. IEEE CVPR, 2001.

[14] S. Ren, K. He, R. Girshick and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," in Advances in Neural Information Processing Systems (NeurIPS), 2015.

[15] W. Liu et al., "SSD: Single shot multibox detector," in Proc. European Conf. Computer Vision (ECCV), 2016.

[16] Y. Zhang, D. Zhou, S. Chen, S. Gao and Y. Ma, "Single-image crowd counting via multi-column convolutional neural network," in Proc. IEEE CVPR, 2016.

[17] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in Proc. IEEE CVPR, 2018.

[18] Y. Li, X. Zhang and D. Chen, "CSRNet: Dilated convolutional neural networks for understanding the highly congested scenes," in Proc. IEEE CVPR, 2018.

[19] T.-Y. Lin et al., "Microsoft COCO: Common objects in context," in Proc. ECCV, 2014.

[20] ungalsoththu, "ChennaiGTFS: Chennai unified GTFS dataset," GitHub repository. [Online]. Available: https://github.com/ungalsoththu/ChennaiGTFS

APPENDIX A – SOURCE CODE

The source code of the two main Python programs is listed in full below: bus_client.py (runs on the Raspberry Pi in each bus) and server.py (central Flask server). The commuter dashboard page (templates/index.html) is shown through its screenshots in Figures 5.1 to 5.3. The files that make up the project folder are listed in Table A.1.

Table A.1: Files of the project

FILE	DESCRIPTION
bus_client.py	Runs on the Raspberry Pi in each bus: captures a photo, counts people with YOLOv8, classifies the crowd level and sends it to the server
server.py	Central Flask server: loads Chennai bus routes from GTFS data, receives the reports of the buses and serves the data to the dashboard
templates/index.html	Bus Crowd Tracker dashboard page for commuters
chennai-unified-gtfs.zip	Chennai public-transport data in GTFS format (route numbers and names)
yolov8n.pt	Pre-trained YOLOv8 nano model weights
test_yolo.py, count_people_yolo.py, app.py	Initial single-camera prototype used for testing detection and the first dashboard
people.jpg	Sample image used for testing detection

A.1 bus_client.py – program on the Raspberry Pi in each bus

```
"""
Runs on the Raspberry Pi INSIDE EACH BUS (one copy per bus).
Takes a photo every CAPTURE_INTERVAL seconds, counts people with YOLO, works out
LOW / MEDIUM / HIGH and sends only that level to the central server.

   pip install ultralytics opencv-python requests      (+ picamera2 for Pi Camera Module)
   python3 bus_client.py
"""
import time

import cv2
import numpy as np
import requests
from ultralytics import YOLO

# ---------------------------------------------------------------------------
# SET THESE FOR EACH BUS
# ---------------------------------------------------------------------------
BUS_ID = "TN01N1234"                        # unique per vehicle (e.g. its registration number)
ROUTE_NO = "21G"                            # the Chennai route this bus is running today
SERVER_URL = "http://192.168.1.10:5000"         # address of the machine running server.py
API_KEY = "change-me"                       # same as BUS_API_KEY on the server
```

```
USE_PICAMERA2 = False       # True = Pi Camera Module (ribbon cable)
CAMERA_INDEX = 0            # USB webcam: 0 is usually right (check: ls /dev/video*)

SEATING_CAPACITY = 40
STANDING_CAPACITY = 30
DETECTION_CORRECTION = 1.0   # see earlier note: >1 if the camera misses people
CAPTURE_INTERVAL = 10       # seconds between photos
# ------------------------------------------------------------------------------
MODEL_PATH = "yolov8n.pt"
CONF_THRESHOLD = 0.4
PERSON_CLASS_ID = 0
WARMUP_FRAMES = 5
FRAME_WIDTH, FRAME_HEIGHT = 640, 480
ROI_POLYGON = None

LOW_MAX = round(SEATING_CAPACITY * 0.75)
MEDIUM_MAX = SEATING_CAPACITY + round(STANDING_CAPACITY * 0.5)

model = YOLO(MODEL_PATH)


def classify_level(count):
    if count <= LOW_MAX:
        return "LOW"
    if count <= MEDIUM_MAX:
        return "MEDIUM"
    return "HIGH"


def in_roi(point):
    if not ROI_POLYGON:
        return True
    return cv2.pointPolygonTest(np.array(ROI_POLYGON, dtype=np.int32), point, False) >= 0


def capture_photo():
    if USE_PICAMERA2:
        from picamera2 import Picamera2
        cam = Picamera2()
        try:
            cam.configure(cam.create_still_configuration(
                main={"size": (FRAME_WIDTH, FRAME_HEIGHT), "format": "RGB888"}))
            cam.start()
            time.sleep(0.5)
            return cam.capture_array()
        finally:
            cam.stop()
            cam.close()
    cap = cv2.VideoCapture(CAMERA_INDEX)
    cap.set(cv2.CAP_PROP_FRAME_WIDTH, FRAME_WIDTH)
    cap.set(cv2.CAP_PROP_FRAME_HEIGHT, FRAME_HEIGHT)
    try:
        frame = None
        for _ in range(WARMUP_FRAMES):
            ok, f = cap.read()
            if ok:
                frame = f
        return frame
```

```
    finally:
        cap.release()


def count_people(frame):
    res = model(frame, conf=CONF_THRESHOLD, classes=[PERSON_CLASS_ID], verbose=False)[0]
    n = 0
    for box in res.boxes:
        x1, y1, x2, y2 = box.xyxy[0].tolist()
        if in_roi(((x1 + x2) / 2, (y1 + y2) / 2)):
            n += 1
    return n


def send(level):
    r = requests.post(f"{SERVER_URL}/api/report",
                      json={"bus_id": BUS_ID, "route": ROUTE_NO, "level": level},
                      headers={"X-API-Key": API_KEY}, timeout=5)
    r.raise_for_status()


while True:
    started = time.time()
    try:
        frame = capture_photo()
        if frame is None:
            raise RuntimeError("camera returned no frame")
        raw = count_people(frame)
        count = round(raw * DETECTION_CORRECTION)
        level = classify_level(count)
        print(f"[{time.strftime('%H:%M:%S')}] detected={raw} estimated={count} -> {level}")
        send(level)
    except requests.RequestException as e:
        print("Could not reach server (will retry next cycle):", e)
    except Exception as e:
        print("Capture failed:", e)
    time.sleep(max(1.0, CAPTURE_INTERVAL - (time.time() - started)))
```

A.2 server.py – central Flask server

```
"""
Central server - runs ONCE (laptop, college server or cloud VM).

* Loads every Chennai bus route from a GTFS file (chennai-unified-gtfs.zip), so
  you never type route details by hand.
* Each bus Pi reports its own bus ID, its route number and its crowd level.
  A bus is registered automatically the first time it reports.
* The web page lists all reporting buses and lets commuters search any route.

    pip install flask
    1. Download the GTFS zip into this folder (see README note below).
    2. BUS_API_KEY=my-secret python3 server.py     (Windows: set BUS_API_KEY=my-secret)

Download (browser, or: curl -L -o chennai-unified-gtfs.zip <url>):
    https://raw.githubusercontent.com/ungalsoththu/ChennaiGTFS/main/data/chennai-unified-gtfs.zip
Check the dataset's licence terms before using it beyond your project.
"""
```

```
import csv
import io
import os
import re
import time
import zipfile
import threading
from collections import deque

from flask import Flask, render_template, jsonify, request

API_KEY = os.environ.get("BUS_API_KEY", "change-me")      # same key on every bus
GTFS_PATH = os.environ.get("GTFS_PATH", os.path.join(os.path.dirname(os.path.abspath(__file__)), "chennai-unified-gtfs.zip"))
OFFLINE_AFTER = 120        # seconds without a report -> "No reading"
REMOVE_AFTER = 600         # seconds without a report -> bus disappears from the list

LEVELS = ("LOW", "MEDIUM", "HIGH")
ID_RE = re.compile(r"^[A-Za-z0-9 _-]{1,30}$")

app = Flask(__name__)
lock = threading.Lock()
buses = {}                 # bus_id -> {"route", "level", "time", "ts", "history"}
ROUTES = {}                # "21G" -> {"route": "21G", "name": "Mogappair West - Redhills"}


def norm_route(r):
    return str(r or "").strip().upper().replace(" ", "")


def load_gtfs(path):
    """Read routes.txt from the GTFS zip and keep the bus routes."""
    if not os.path.exists(path):
        print(f"[warn] GTFS file not found at {path}.\n"
              "       Running in open mode: any route number is accepted and shown without a name.")
        return
    with zipfile.ZipFile(path) as z:
        name = next((n for n in z.namelist() if n.lower().endswith("routes.txt")), None)
        if not name:
            print("[warn] routes.txt not found inside the GTFS zip. Running in open mode.")
            return
        with z.open(name) as f:
            for row in csv.DictReader(io.TextIOWrapper(f, encoding="utf-8-sig")):
                rtype = (row.get("route_type") or "").strip()
                is_bus = rtype == "" or rtype == "3" or (rtype.isdigit() and 700 <= int(rtype) <= 799)
                if not is_bus:                    # skip metro / train lines
                    continue
                short = (row.get("route_short_name") or "").strip()
                long_ = (row.get("route_long_name") or "").strip()
                rid = (row.get("route_id") or "").strip()
                key = norm_route(short or rid)
                if key and key not in ROUTES:
                    ROUTES[key] = {"route": key, "name": long_ or short or rid}
    print(f"[info] Loaded {len(ROUTES)} bus routes from GTFS.")


@app.route("/")
def index():
    return render_template("index.html")
```

```
@app.route("/api/report", methods=["POST"])
def report():
    """Called by each bus every few seconds: {bus_id, route, level}."""
    if request.headers.get("X-API-Key") != API_KEY:
        return jsonify(error="Bad API key"), 401
    body = request.get_json(silent=True) or {}
    bus_id, level = str(body.get("bus_id", "")).strip(), body.get("level")
    route = norm_route(body.get("route"))
    if not ID_RE.match(bus_id):
        return jsonify(error="bus_id must be 1-30 letters, digits, space, - or _"), 400
    if not route:
        return jsonify(error="route is required"), 400
    if ROUTES and route not in ROUTES:
        return jsonify(error=f"Unknown Chennai bus route '{route}'"), 404
    if level not in LEVELS:
        return jsonify(error="level must be LOW, MEDIUM or HIGH"), 400
    stamp = time.strftime("%H:%M:%S")
    with lock:
        b = buses.setdefault(bus_id, {"history": deque(maxlen=60)})
        b.update(route=route, level=level, time=stamp, ts=time.time())
        b["history"].append({"t": stamp, "level": level})
    return jsonify(ok=True)


@app.route("/api/buses")
def list_buses():
    now, out = time.time(), []
    with lock:
        for bus_id in [k for k, b in buses.items() if now - b["ts"] > REMOVE_AFTER]:
            del buses[bus_id]
        for bus_id, b in buses.items():
            age = int(now - b["ts"])
            info = ROUTES.get(b["route"])
            out.append({
                "id": bus_id, "route": b["route"],
                "dest": info["name"] if info else "Route " + b["route"],
                "level": b["level"] if age <= OFFLINE_AFTER else "OFFLINE",
                "time": b["time"], "age": age,
            })
    return jsonify(out)


@app.route("/api/routes")
def search_routes():
    """Search every Chennai route by number or name, with how many buses are live on it."""
    q = norm_route(request.args.get("q", ""))
    if not q or not ROUTES:
        return jsonify([])
    now = time.time()
    with lock:
        live = {}
        for b in buses.values():
            if now - b["ts"] <= OFFLINE_AFTER:
                live[b["route"]] = live.get(b["route"], 0) + 1
    starts = [r for k, r in ROUTES.items() if k.startswith(q)]
    contains = [r for k, r in ROUTES.items()
```

```
            if not k.startswith(q) and (q in k or q in r["name"].upper().replace(" ", ""))]
    starts.sort(key=lambda r: (len(r["route"]), r["route"]))
    return jsonify([{**r, "live": live.get(r["route"], 0)} for r in (starts + contains)[:15]])


@app.route("/api/history/<bus_id>")
def history(bus_id):
    with lock:
        return jsonify(list(buses.get(bus_id, {}).get("history", [])))


if __name__ == "__main__":
    load_gtfs(GTFS_PATH)
    app.run(host="0.0.0.0", port=5000, threaded=True)
```

APPENDIX B – CONFERENCE ACCEPTANCE LETTER

Abstract acceptance letter received from the IMCCRT-2026 Secretariat on 12 October 2026 (Abstract ID: IMCCRT-2026-2578).

Abstract Acceptance Letter-Abstract ID: IMCCRT-2026-2578
1 message

IMCCRT 2026 <imccrt2026@gmail.com> Mon, 12 Oct, 2026 at 11:52 am
To: dharan <evbot2007@gmail.com>

Congratulations!!!

We are pleased to inform you that your abstract title has been accepted for **"CROWD DETECTION IN A BUS"** in the **13th International Multidisciplinary Conference on Current Research Trends-2026 (IMCCRT-2026) on 30th October 2026 in the online platform (Virtual Presentation).** To confirm your registration, kindly pay your conference fee as per the invoice given in page 2.

Please mention your Abstract ID (IMCCRT-2026-2578) when you do your payment transaction.

All accepted abstracts will be added in the conference proceeding with ISBN number.

Please submit your full paper for review on imccrt2026@gmail.com

Thank you very much for your kind participation. Please do not hesitate to contact us if you need assistance.

Best regards,

IMCCRT- 2026 Secretariat
Contact Email: imccrt2026@gmail.com
Website: https://imccrt.iikr.in/index.html

2 attachments

IMCCRT-2026-2578.pdf
384 KB

PPT 13th IMCCRT.pptx
887 KB

Abstract acceptance letter – 13th International Multidisciplinary Conference on Current Research Trends (IMCCRT-2026)